Module 6: Introduction to Generative Adversarial Networks (GANs)

Balaji Srinivasan, Ganapathy Krishnamurthi

Wadhwani School of AI , IIT Madras

Outline of this Module

1. Introduction to Generative Models
2. Basics of Generative models
3. Introduction to GANs

Introduction to Generative Models

Discriminative models and Generative models

What are Generative models

Definition

A **generative model** is a type of statistical model that is capable of learning the underlying **distribution of a dataset** in order to generate new, synthetic data samples that are similar to the original data.

What are Generative models

Definition

A **generative model** is a type of statistical model that is capable of learning the underlying **distribution of a dataset** in order to generate new, synthetic data samples that are similar to the original data.

Key Features

- The core idea is to understand the *process* by which the data originated.

What are Generative models

Definition

A **generative model** is a type of statistical model that is capable of learning the underlying **distribution of a dataset** in order to generate new, synthetic data samples that are similar to the original data.

Key Features

- The core idea is to understand the *process* by which the data originated.
- Models the **joint probability distribution** $P(X, Y)$ of the features and labels.

What are Generative models

Definition

A **generative model** is a type of statistical model that is capable of learning the underlying **distribution of a dataset** in order to generate new, synthetic data samples that are similar to the original data.

Key Features

- The core idea is to understand the *process* by which the data originated.
- Models the **joint probability distribution** $P(X, Y)$ of the features and labels.
- Can be used to **generate new data samples** that are not present in the original dataset.

What are Generative models

Definition

A **generative model** is a type of statistical model that is capable of learning the underlying **distribution of a dataset** in order to generate new, synthetic data samples that are similar to the original data.

Key Features

- The core idea is to understand the *process* by which the data originated.
- Models the **joint probability distribution** $P(X, Y)$ of the features and labels.
- Can be used to **generate new data samples** that are not present in the original dataset.
- Often learns a compressed and meaningful **latent space representation** of the data.

Why are they powerful?

Capabilities of Generative models

- **Data Generation :** Create new, realistic samples.

Why are they powerful?

Capabilities of Generative models

- **Data Generation :** Create new, realistic samples.
- **Understanding Data:** Reveal latent structures and dependencies within the data.

Why are they powerful?

Capabilities of Generative models

- **Data Generation :** Create new, realistic samples.
- **Understanding Data:** Reveal latent structures and dependencies within the data.
- **Missing Data Imputation:** Fill in gaps in incomplete data.

Why are they powerful?

Capabilities of Generative models

- **Data Generation :** Create new, realistic samples.
- **Understanding Data:** Reveal latent structures and dependencies within the data.
- **Missing Data Imputation:** Fill in gaps in incomplete data.
- **Anomaly Detection:** Identify samples that deviate from the learned distribution.

Why are they powerful?

Capabilities of Generative models

- **Data Generation :** Create new, realistic samples.
- **Understanding Data:** Reveal latent structures and dependencies within the data.
- **Missing Data Imputation:** Fill in gaps in incomplete data.
- **Anomaly Detection:** Identify samples that deviate from the learned distribution.
- **Representation Learning:** Discover meaningful low-dimensional representations of the data.

Toy Example: 2x2 Image Grid

- Let us consider a very simple "image" as a 2x2 grid of pixels.

Toy Example: 2x2 Image Grid

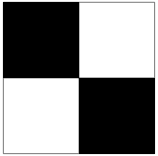
- Let us consider a very simple "image" as a 2x2 grid of pixels.
- Each pixel can be black (0) or white (1).

Toy Example: 2x2 Image Grid

- Let us consider a very simple "image" as a 2x2 grid of pixels.
- Each pixel can be black (0) or white (1).
- This means that there are $2^{2\times 2} = 2^4 = 16$ possible images.

Toy Example: 2x2 Image Grid

- Let us consider a very simple "image" as a 2x2 grid of pixels.
- Each pixel can be black (0) or white (1).
- This means that there are $2^{2\times 2} = 2^4 = 16$ possible images.



Example Image

Learning the Distribution for a 2x2 Grid

- Imagine we have a dataset of 2x2 images.

Learning the Distribution for a 2x2 Grid

- Imagine we have a dataset of 2x2 images.
- A generative model would try to learn $P(x_{11}, x_{12}, x_{21}, x_{22})$, where x_{ij} is the pixel value at row i, column j.

Learning the Distribution for a 2x2 Grid

- Imagine we have a dataset of 2x2 images.
- A generative model would try to learn $P(x_{11}, x_{12}, x_{21}, x_{22})$, where x_{ij} is the pixel value at row i, column j.
- In this discrete case, we can simply count the occurrences of each of the 16 possible images and normalize to get probabilities.

Representing a 2x2 Image Distribution

- For a small grid like 2x2, we could even explicitly list the probability for each of the 16 configurations.

Representing a 2x2 Image Distribution

- For a small grid like 2x2, we could even explicitly list the probability for each of the 16 configurations.
- Example:
  - $P(\text{all white}) = 0.3$
  - $P(\text{diagonal black}) = 0.2$
  - $P(\text{checkerboard}) = 0.1$
  - ... and so on for all 16 states.

Representing a 2x2 Image Distribution

- For a small grid like 2x2, we could even explicitly list the probability for each of the 16 configurations.
- Example:
  - $P(\text{all white}) = 0.3$
  - $P(\text{diagonal black}) = 0.2$
  - $P(\text{checkerboard}) = 0.1$
  - ... and so on for all 16 states.
- For real images, the number of pixels is huge ($256 \times 256 \times 3$ for color images), making explicit enumeration impossible. Generative models use clever techniques to approximate this high-dimensional distribution.

Application 1: Generating New Samples

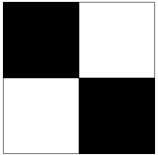
- Once the generative model has learned the distribution $P(\mathbf{x})$, we can sample from it to create new data points.

Application 1: Generating New Samples

- Once the generative model has learned the distribution $P(\mathbf{x})$, we can sample from it to create new data points.
- For our 2x2 grid example: If we learned that the "diagonal black" pattern is common, the model would be more likely to generate such an image.

Application 1: Generating New Samples

- Once the generative model has learned the distribution $P(\mathbf{x})$, we can sample from it to create new data points.
- For our 2x2 grid example: If we learned that the "diagonal black" pattern is common, the model would be more likely to generate such an image.



Sampled Image 1

Application 2: Marginalization - Intuition and Example

Numerical Example (2x2 Grid):

- Let x_{11} be the top-left pixel (1=white, 0=black).

Application 2: Marginalization - Intuition and Example

Numerical Example (2x2 Grid):

- Let x_{11} be the top-left pixel (1=white, 0=black).
- We want $P(x_{11} = 1)$ - the probability that the top-left pixel is white.

Application 2: Marginalization - Intuition and Example

Numerical Example (2x2 Grid):

- Let x_{11} be the top-left pixel (1=white, 0=black).
- We want $P(x_{11} = 1)$ - the probability that the top-left pixel is white.
- We need to sum the probabilities of all 8 possible images where $x_{11} = 1$.

Application 2: Marginalization - Intuition and Example

Numerical Example (2x2 Grid):

- Let x_{11} be the top-left pixel (1=white, 0=black).
- We want $P(x_{11} = 1)$ - the probability that the top-left pixel is white.
- We need to sum the probabilities of all 8 possible images where $x_{11} = 1$.

Hypothetical Joint Probabilities:

- $P(x_{11} = 1, x_{12} = 0, x_{21} = 0, x_{22} = 0) = 0.05$
- $P(x_{11} = 1, x_{12} = 0, x_{21} = 0, x_{22} = 1) = 0.10$
- $P(x_{11} = 1, x_{12} = 0, x_{21} = 1, x_{22} = 0) = 0.15$
- $P(x_{11} = 1, x_{12} = 0, x_{21} = 1, x_{22} = 1) = 0.03$
- $P(x_{11} = 1, x_{12} = 1, x_{21} = 0, x_{22} = 0) = 0.02$
- $P(x_{11} = 1, x_{12} = 1, x_{21} = 0, x_{22} = 1) = 0.08$
- $P(x_{11} = 1, x_{12} = 1, x_{21} = 1, x_{22} = 0) = 0.07$
- $P(x_{11} = 1, x_{12} = 1, x_{21} = 1, x_{22} = 1) = 0.01$

Application 2: Marginalization

- Marginalization means finding the probability of a subset of variables by summing (or integrating) over the others.

Application 2: Marginalization

- Marginalization means finding the probability of a subset of variables by summing (or integrating) over the others.
- If we have $P(x_1, x_2, x_3, x_4)$ for our 2x2 grid, we can find the probability of $P(x_1)$ (e.g., the top-left pixel being black) by summing over all possible states of x_2, x_3, x_4:

$$P(x_1) = \sum_{x_2} \sum_{x_3} \sum_{x_4} P(x_1, x_2, x_3, x_4)$$

Application 2: Marginalization

- Marginalization means finding the probability of a subset of variables by summing (or integrating) over the others.
- If we have $P(x_1, x_2, x_3, x_4)$ for our 2x2 grid, we can find the probability of $P(x_1)$ (e.g., the top-left pixel being black) by summing over all possible states of x_2, x_3, x_4:

$$P(x_1) = \sum_{x_2} \sum_{x_3} \sum_{x_4} P(x_1, x_2, x_3, x_4)$$

- This tells us the overall likelihood of a specific pixel value, irrespective of other pixels.

Application 3: In-painting / Missing Data Imputation

- If parts of an image are missing (e.g., some pixels are unknown), a generative model can "fill them in".

Application 3: In-painting / Missing Data Imputation

- If parts of an image are missing (e.g., some pixels are unknown), a generative model can "fill them in".
- This is done by conditioning on the known pixels and generating the most probable values for the missing ones.

Application 3: In-painting / Missing Data Imputation

- If parts of an image are missing (e.g., some pixels are unknown), a generative model can "fill them in".
- This is done by conditioning on the known pixels and generating the most probable values for the missing ones.
- Given a partially observed image $\mathbf{x}_{obs}$ and missing pixels $\mathbf{x}_{mis}$, we want to find $P(\mathbf{x}_{mis}|\mathbf{x}_{obs})$.

Application 3: In-painting / Missing Data Imputation

- If parts of an image are missing (e.g., some pixels are unknown), a generative model can "fill them in".
- This is done by conditioning on the known pixels and generating the most probable values for the missing ones.
- Given a partially observed image $\mathbf{x}_{obs}$ and missing pixels $\mathbf{x}_{mis}$, we want to find $P(\mathbf{x}_{mis}|\mathbf{x}_{obs})$.
- Using Bayes' theorem and the learned joint distribution:

$$P(\mathbf{x}_{mis}|\mathbf{x}_{obs}) = \frac{P(\mathbf{x}_{mis}, \mathbf{x}_{obs})}{P(\mathbf{x}_{obs})} = \frac{P(\mathbf{x}_{mis}, \mathbf{x}_{obs})}{\sum_{\mathbf{x}'_{mis}} P(\mathbf{x}'_{mis}, \mathbf{x}_{obs})}$$

Application 3: In-painting / Missing Data Imputation

- If parts of an image are missing (e.g., some pixels are unknown), a generative model can "fill them in".
- This is done by conditioning on the known pixels and generating the most probable values for the missing ones.
- Given a partially observed image $\mathbf{x}_{obs}$ and missing pixels $\mathbf{x}_{mis}$, we want to find $P(\mathbf{x}_{mis}|\mathbf{x}_{obs})$.
- Using Bayes' theorem and the learned joint distribution:

$$P(\mathbf{x}_{mis}|\mathbf{x}_{obs}) = \frac{P(\mathbf{x}_{mis}, \mathbf{x}_{obs})}{P(\mathbf{x}_{obs})} = \frac{P(\mathbf{x}_{mis}, \mathbf{x}_{obs})}{\sum_{\mathbf{x}'_{mis}} P(\mathbf{x}'_{mis}, \mathbf{x}_{obs})}$$

- For our 2x2 grid, if we know the top-left pixel is black and others are unknown, the model can predict the most likely full image.

In-painting Example: 2x2 Grid (1/3)

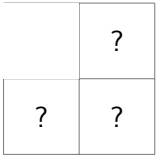
- Let's assume we have learned the full joint distribution for all 16 possible 2x2 binary images.

In-painting Example: 2x2 Grid (1/3)

- Let's assume we have learned the full joint distribution for all 16 possible 2x2 binary images.
- Our input image has one known pixel ($x_{11} = 1$, top-left is white) and three missing pixels (x_{12}, x_{21}, x_{22}).

In-painting Example: 2x2 Grid (1/3)

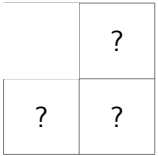
- Let's assume we have learned the full joint distribution for all 16 possible 2x2 binary images.
- Our input image has one known pixel ($x_{11} = 1$, top-left is white) and three missing pixels (x_{12}, x_{21}, x_{22}).



Input (Partial)

In-painting Example: 2x2 Grid (1/3)

- Let's assume we have learned the full joint distribution for all 16 possible 2x2 binary images.
- Our input image has one known pixel ($x_{11} = 1$, top-left is white) and three missing pixels (x_{12}, x_{21}, x_{22}).

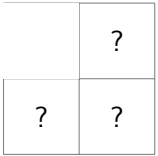


Input (Partial)

- We want to find the most probable configuration for (x_{12}, x_{21}, x_{22}) given $x_{11} = 1$.

In-painting Example: 2x2 Grid (1/3)

- Let's assume we have learned the full joint distribution for all 16 possible 2x2 binary images.
- Our input image has one known pixel ($x_{11} = 1$, top-left is white) and three missing pixels (x_{12}, x_{21}, x_{22}).



Input (Partial)

- We want to find the most probable configuration for (x_{12}, x_{21}, x_{22}) given $x_{11} = 1$.
- This means we need to calculate $P(x_{12}, x_{21}, x_{22} | x_{11} = 1)$ for all $2^3 = 8$ combinations of the missing pixels.

In-painting Example: 2x2 Grid (2/3)

- Recall $P(\mathbf{x}_{mis}|\mathbf{x}_{obs}) = \frac{P(\mathbf{x}_{mis},\mathbf{x}_{obs})}{\sum_{\mathbf{x}'_{mis}} P(\mathbf{x}'_{mis},\mathbf{x}_{obs})}$.

In-painting Example: 2x2 Grid (2/3)

- Recall $P(\mathbf{x}_{mis}|\mathbf{x}_{obs}) = \frac{P(\mathbf{x}_{mis},\mathbf{x}_{obs})}{\sum_{\mathbf{x}'_{mis}} P(\mathbf{x}'_{mis},\mathbf{x}_{obs})}$.
- The denominator $\sum_{\mathbf{x}'_{mis}} P(\mathbf{x}'_{mis},\mathbf{x}_{obs})$ is $P(\mathbf{x}_{obs})$, which is constant for all candidates.

In-painting Example: 2x2 Grid (2/3)

- Recall $P(\mathbf{x}_{mis}|\mathbf{x}_{obs}) = \frac{P(\mathbf{x}_{mis},\mathbf{x}_{obs})}{\sum_{\mathbf{x}'_{mis}} P(\mathbf{x}'_{mis},\mathbf{x}_{obs})}$.
- The denominator $\sum_{\mathbf{x}'_{mis}} P(\mathbf{x}'_{mis}, \mathbf{x}_{obs})$ is $P(\mathbf{x}_{obs})$, which is constant for all candidates.
- So, we just need to find $\text{argmax}_{\mathbf{x}_{mis}} P(\mathbf{x}_{mis}, \mathbf{x}_{obs})$.

In-painting Example: 2x2 Grid (2/3)

- Recall $P(\mathbf{x}_{mis}|\mathbf{x}_{obs}) = \frac{P(\mathbf{x}_{mis},\mathbf{x}_{obs})}{\sum_{\mathbf{x}'_{mis}} P(\mathbf{x}'_{mis},\mathbf{x}_{obs})}$.
- The denominator $\sum_{\mathbf{x}'_{mis}} P(\mathbf{x}'_{mis}, \mathbf{x}_{obs})$ is $P(\mathbf{x}_{obs})$, which is constant for all candidates.
- So, we just need to find $\text{argmax}_{\mathbf{x}_{mis}} P(\mathbf{x}_{mis}, \mathbf{x}_{obs})$.
- Let's use a hypothetical learned joint distribution $P(x_{11}, x_{12}, x_{21}, x_{22})$:

- $P(1,0,0,0) = 0.05$
- $P(1,0,0,1) = 0.10$
- $P(1,0,1,0) = 0.15$
- $P(1,0,1,1) = 0.03$
- $P(1,1,0,0) = 0.02$
- $P(1,1,0,1) = 0.08$
- $P(1,1,1,0) = 0.07$
- $P(1,1,1,1) = 0.01$

In-painting Example: 2x2 Grid (2/3)

- Recall $P(\mathbf{x}_{mis}|\mathbf{x}_{obs}) = \frac{P(\mathbf{x}_{mis},\mathbf{x}_{obs})}{\sum_{\mathbf{x}'_{mis}} P(\mathbf{x}'_{mis},\mathbf{x}_{obs})}$.
- The denominator $\sum_{\mathbf{x}'_{mis}} P(\mathbf{x}'_{mis}, \mathbf{x}_{obs})$ is $P(\mathbf{x}_{obs})$, which is constant for all candidates.
- So, we just need to find $\text{argmax}_{\mathbf{x}_{mis}} P(\mathbf{x}_{mis}, \mathbf{x}_{obs})$.
- Let's use a hypothetical learned joint distribution $P(x_{11}, x_{12}, x_{21}, x_{22})$:

- $P(1,0,0,0) = 0.05$
- $P(1,0,0,1) = 0.10$
- $P(1,0,1,0) = 0.15$
- $P(1,0,1,1) = 0.03$
- $P(1,1,0,0) = 0.02$
- $P(1,1,0,1) = 0.08$
- $P(1,1,1,0) = 0.07$
- $P(1,1,1,1) = 0.01$

- ... and probabilities for the other 8 images where $x_{11} = 0$.

In-painting Example: 2x2 Grid (2/3)

- Recall $P(\mathbf{x}_{mis}|\mathbf{x}_{obs}) = \frac{P(\mathbf{x}_{mis},\mathbf{x}_{obs})}{\sum_{\mathbf{x}'_{mis}} P(\mathbf{x}'_{mis},\mathbf{x}_{obs})}$.
- The denominator $\sum_{\mathbf{x}'_{mis}} P(\mathbf{x}'_{mis}, \mathbf{x}_{obs})$ is $P(\mathbf{x}_{obs})$, which is constant for all candidates.
- So, we just need to find $\text{argmax}_{\mathbf{x}_{mis}} P(\mathbf{x}_{mis}, \mathbf{x}_{obs})$.
- Let's use a hypothetical learned joint distribution $P(x_{11}, x_{12}, x_{21}, x_{22})$:

- $P(1,0,0,0) = 0.05$
- $P(1,0,0,1) = 0.10$
- $P(1,0,1,0) = 0.15$
- $P(1,0,1,1) = 0.03$
- $P(1,1,0,0) = 0.02$
- $P(1,1,0,1) = 0.08$
- $P(1,1,1,0) = 0.07$
- $P(1,1,1,1) = 0.01$

- ... and probabilities for the other 8 images where $x_{11} = 0$.
- We are looking for (x_{12}, x_{21}, x_{22}) that maximizes $P(1, x_{12}, x_{21}, x_{22})$.

In-painting Example: 2x2 Grid (2/3)

- Recall $P(\mathbf{x}_{mis}|\mathbf{x}_{obs}) = \frac{P(\mathbf{x}_{mis},\mathbf{x}_{obs})}{\sum_{\mathbf{x}'_{mis}} P(\mathbf{x}'_{mis},\mathbf{x}_{obs})}$.
- The denominator $\sum_{\mathbf{x}'_{mis}} P(\mathbf{x}'_{mis}, \mathbf{x}_{obs})$ is $P(\mathbf{x}_{obs})$, which is constant for all candidates.
- So, we just need to find $\text{argmax}_{\mathbf{x}_{mis}} P(\mathbf{x}_{mis}, \mathbf{x}_{obs})$.
- Let's use a hypothetical learned joint distribution $P(x_{11}, x_{12}, x_{21}, x_{22})$:

- $P(1,0,0,0) = 0.05$
- $P(1,0,0,1) = 0.10$
- $P(1,0,1,0) = 0.15$
- $P(1,0,1,1) = 0.03$
- $P(1,1,0,0) = 0.02$
- $P(1,1,0,1) = 0.08$
- $P(1,1,1,0) = 0.07$
- $P(1,1,1,1) = 0.01$

- ... and probabilities for the other 8 images where $x_{11} = 0$.
- We are looking for (x_{12}, x_{21}, x_{22}) that maximizes $P(1, x_{12}, x_{21}, x_{22})$.
- From our list, $P(1,0,1,0) = 0.15$ is the maximum.

In-painting Example: 2x2 Grid (3/3)

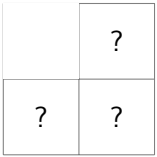
- The most probable completion for (x_{12}, x_{21}, x_{22}) is $(0, 1, 0)$.

In-painting Example: 2x2 Grid (3/3)

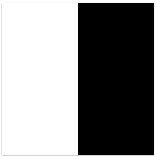
- The most probable completion for (x_{12}, x_{21}, x_{22}) is $(0, 1, 0)$.
- This means $x_{12} = 0$ (black), $x_{21} = 1$ (white), $x_{22} = 0$ (black).

In-painting Example: 2x2 Grid (3/3)

- The most probable completion for (x_{12}, x_{21}, x_{22}) is $(0, 1, 0)$.
- This means $x_{12} = 0$ (black), $x_{21} = 1$ (white), $x_{22} = 0$ (black).



Input (Partial) $\xrightarrow{\text{Generative Model}}$ Output (In-painted)



In-painting Example: 2x2 Grid (3/3)

- The most probable completion for (x_{12}, x_{21}, x_{22}) is $(0, 1, 0)$.
- This means $x_{12} = 0$ (black), $x_{21} = 1$ (white), $x_{22} = 0$ (black).

- The model fills in the '?' based on the learned distribution, making the most probable completion.

Computational Bottleneck: Dimension Increase

- Our 2x2 grid has 4 pixels. If 3 are missing, we check $2^3 = 8$ combinations. This is trivial.

Computational Bottleneck: Dimension Increase

- Our 2x2 grid has 4 pixels. If 3 are missing, we check $2^3 = 8$ combinations. This is trivial.
- Consider a small real-world image: a 32x32 grayscale image.
  - Total pixels: $32 \times 32 = 1024$.
  - Number of possible images: 2^{1024} (if binary pixels) â€“ astronomically large.

Computational Bottleneck: Dimension Increase

- Our 2x2 grid has 4 pixels. If 3 are missing, we check $2^3 = 8$ combinations. This is trivial.
- Consider a small real-world image: a 32x32 grayscale image.
  - Total pixels: $32 \times 32 = 1024$.
  - Number of possible images: 2^{1024} (if binary pixels) â€“ astronomically large.
- If even half the pixels are missing ($1024/2 = 512$ pixels), we would need to sum over 2^{512} possibilities for the denominator or explicitly check 2^{512} candidates for the numerator.

Computational Bottleneck: Dimension Increase

- Our 2x2 grid has 4 pixels. If 3 are missing, we check $2^3 = 8$ combinations. This is trivial.
- Consider a small real-world image: a 32x32 grayscale image.
  - Total pixels: $32 \times 32 = 1024$.
  - Number of possible images: 2^{1024} (if binary pixels) â€“ astronomically large.
- If even half the pixels are missing ($1024/2 = 512$ pixels), we would need to sum over 2^{512} possibilities for the denominator or explicitly check 2^{512} candidates for the numerator.
- This is computationally impossible for explicit generative models.

How Modern Models Address the Bottleneck

- Modern generative models (GANs, VAEs, Diffusion Models) do not explicitly learn or store the full joint probability table.

How Modern Models Address the Bottleneck

- Modern generative models (GANs, VAEs, Diffusion Models) do not explicitly learn or store the full joint probability table.
- Instead, they learn a function that can approximate this distribution or generate samples from it.

How Modern Models Address the Bottleneck

- Modern generative models (GANs, VAEs, Diffusion Models) do not explicitly learn or store the full joint probability table.
- Instead, they learn a function that can approximate this distribution or generate samples from it.
- For in-painting, they use these learned functions to:
  - **Sample iteratively:** Start with random values for missing pixels and refine them.

How Modern Models Address the Bottleneck

- Modern generative models (GANs, VAEs, Diffusion Models) do not explicitly learn or store the full joint probability table.
- Instead, they learn a function that can approximate this distribution or generate samples from it.
- For in-painting, they use these learned functions to:
  - **Sample iteratively:** Start with random values for missing pixels and refine them.
  - **Directly predict:** A neural network (e.g., U-Net in Diffusion Models) is trained to directly predict the missing pixels given the context.

How Modern Models Address the Bottleneck

- Modern generative models (GANs, VAEs, Diffusion Models) do not explicitly learn or store the full joint probability table.
- Instead, they learn a function that can approximate this distribution or generate samples from it.
- For in-painting, they use these learned functions to:
  - **Sample iteratively:** Start with random values for missing pixels and refine them.
  - **Directly predict:** A neural network (e.g., U-Net in Diffusion Models) is trained to directly predict the missing pixels given the context.
  - **Latent space manipulation:** For VAEs and GANs, in-painting can involve encoding the partial image into a latent space, manipulating the latent representation, and decoding it.

How Modern Models Address the Bottleneck

- Modern generative models (GANs, VAEs, Diffusion Models) do not explicitly learn or store the full joint probability table.
- Instead, they learn a function that can approximate this distribution or generate samples from it.
- For in-painting, they use these learned functions to:
  - **Sample iteratively:** Start with random values for missing pixels and refine them.
  - **Directly predict:** A neural network (e.g., U-Net in Diffusion Models) is trained to directly predict the missing pixels given the context.
  - **Latent space manipulation:** For VAEs and GANs, in-painting can involve encoding the partial image into a latent space, manipulating the latent representation, and decoding it.
- These methods avoid the explicit enumeration of all possible pixel combinations, making in-painting feasible for high-resolution images.

Application 4: Classification (with a known distribution)

- While primarily for generation, if we have generative models for each class $P(\mathbf{x}|y = c)$, we can use them for classification.

Application 4: Classification (with a known distribution)

- While primarily for generation, if we have generative models for each class $P(\mathbf{x}|y = c)$, we can use them for classification.
- Given a new input $\mathbf{x}$, we want to find the most probable class y.

Application 4: Classification (with a known distribution)

- While primarily for generation, if we have generative models for each class $P(\mathbf{x}|y = c)$, we can use them for classification.
- Given a new input $\mathbf{x}$, we want to find the most probable class y.
- Using Bayes' theorem:

$$P(y = c|\mathbf{x}) = \frac{P(\mathbf{x}|y = c)P(y = c)}{P(\mathbf{x})}$$

Application 4: Classification (with a known distribution)

- While primarily for generation, if we have generative models for each class $P(\mathbf{x}|y = c)$, we can use them for classification.
- Given a new input $\mathbf{x}$, we want to find the most probable class y.
- Using Bayes' theorem:

$$P(y = c|\mathbf{x}) = \frac{P(\mathbf{x}|y = c)P(y = c)}{P(\mathbf{x})}$$

- We choose the class c that maximizes $P(\mathbf{x}|y = c)P(y = c)$ (as $P(\mathbf{x})$ is constant for a given $\mathbf{x}$).

Application 4: Classification (with a known distribution)

- While primarily for generation, if we have generative models for each class $P(\mathbf{x}|y = c)$, we can use them for classification.
- Given a new input $\mathbf{x}$, we want to find the most probable class y.
- Using Bayes' theorem:

$$P(y = c|\mathbf{x}) = \frac{P(\mathbf{x}|y = c)P(y = c)}{P(\mathbf{x})}$$

- We choose the class c that maximizes $P(\mathbf{x}|y = c)P(y = c)$ (as $P(\mathbf{x})$ is constant for a given $\mathbf{x}$).
- This is known as a generative classifier.

Generative vs. Discriminative Classifiers

- Generative Classifier: Models $P(\mathbf{x}|y)$ and $P(y)$. Can generate data.
  - Examples: Naive Bayes, Latent Dirichlet Allocation (LDA), some types of Neural Networks.

Generative vs. Discriminative Classifiers

- Generative Classifier: Models $P(\mathbf{x}|y)$ and $P(y)$. Can generate data.
  - Examples: Naive Bayes, Latent Dirichlet Allocation (LDA), some types of Neural Networks.
- Discriminative Classifier: Directly models $P(y|\mathbf{x})$. Learns decision boundaries.
  - Examples: Logistic Regression, Support Vector Machines (SVMs), standard Neural Networks.

Generative vs. Discriminative Classifiers

- Generative Classifier: Models $P(\mathbf{x}|y)$ and $P(y)$. Can generate data.
  - Examples: Naive Bayes, Latent Dirichlet Allocation (LDA), some types of Neural Networks.
- Discriminative Classifier: Directly models $P(y|\mathbf{x})$. Learns decision boundaries.
  - Examples: Logistic Regression, Support Vector Machines (SVMs), standard Neural Networks.
- Generative classifiers can be more robust to missing data and provide richer information, but often require more data to learn $P(\mathbf{x}|y)$ accurately.

Illustrations of Generative Models

Text-to-Video

OpenAI's Sora 2: A cutting-edge model that generates high-definition, coherent videos from simple text prompts.

Illustrations of Generative Models

Text-to-Video

OpenAI's Sora 2: A cutting-edge model that generates high-definition, coherent videos from simple text prompts.

Text-to-Image

- **Google's Imagen 2:** A powerful model known for generating highly realistic images and offering advanced editing capabilities like inpainting and outpainting based on text instructions.

Illustrations of Generative Models

Text-to-Video

OpenAI's Sora 2: A cutting-edge model that generates high-definition, coherent videos from simple text prompts.

Text-to-Image

- **Google's Imagen 2:** A powerful model known for generating highly realistic images and offering advanced editing capabilities like inpainting and outpainting based on text instructions.
- **Stable Diffusion XL:** A widely-used open-source model celebrated for producing images with fine detail and aesthetic composition, supported by a large community.

Illustrations of Generative Models

Text-to-Video

OpenAI's Sora 2: A cutting-edge model that generates high-definition, coherent videos from simple text prompts.

Text-to-Image

- **Google's Imagen 2:** A powerful model known for generating highly realistic images and offering advanced editing capabilities like inpainting and outpainting based on text instructions.
- **Stable Diffusion XL:** A widely-used open-source model celebrated for producing images with fine detail and aesthetic composition, supported by a large community.

Text-to-Audio

Meta's AudioGen: A state-of-the-art model that can generate high-fidelity audio samples, including sound effects and simple musical pieces, directly from text descriptions.

Mathematical Formulation

The primary goal of a generative model is to learn the probability distribution $p_{data}(\mathbf{x})$ of the real data $\mathbf{x}$.

Mathematical Formulation

The primary goal of a generative model is to learn the probability distribution $p_{data}(\mathbf{x})$ of the real data $\mathbf{x}$.

The Goal

The model, let's call it G, learns a distribution $p_{model}(\mathbf{x})$. The objective is to make p_{model} as close as possible to p_{data}.

Mathematical Formulation

Often, this is done by transforming a simple noise distribution into the complex data distribution:

Mathematical Formulation

Often, this is done by transforming a simple noise distribution into the complex data distribution:

- Let $\mathbf{z}$ be a random vector sampled from a simple prior distribution, like a Gaussian. $\mathbf{z} \sim p(\mathbf{z})$

Mathematical Formulation

Often, this is done by transforming a simple noise distribution into the complex data distribution:

- Let $\mathbf{z}$ be a random vector sampled from a simple prior distribution, like a Gaussian. $\mathbf{z} \sim p(\mathbf{z})$
- The generative model is a function $G(\mathbf{z}; \theta_g)$ parameterized by θ_g that maps the latent space $\mathcal{Z}$ to the data space $\mathcal{X}$.

Mathematical Formulation

Often, this is done by transforming a simple noise distribution into the complex data distribution:

- Let $\mathbf{z}$ be a random vector sampled from a simple prior distribution, like a Gaussian. $\mathbf{z} \sim p(\mathbf{z})$
- The generative model is a function $G(\mathbf{z}; \theta_g)$ parameterized by θ_g that maps the latent space $\mathcal{Z}$ to the data space $\mathcal{X}$.
- The model generates a sample $\mathbf{x} = G(\mathbf{z})$.

Mathematical Formulation

Often, this is done by transforming a simple noise distribution into the complex data distribution:

- Let $\mathbf{z}$ be a random vector sampled from a simple prior distribution, like a Gaussian. $\mathbf{z} \sim p(\mathbf{z})$
- The generative model is a function $G(\mathbf{z}; \theta_g)$ parameterized by θ_g that maps the latent space $\mathcal{Z}$ to the data space $\mathcal{X}$.
- The model generates a sample $\mathbf{x} = G(\mathbf{z})$.
- The training process aims to find the optimal parameters θ_g^* such that the distribution of $G(\mathbf{z})$ matches the distribution of real data.

Illustrative example of Generative model

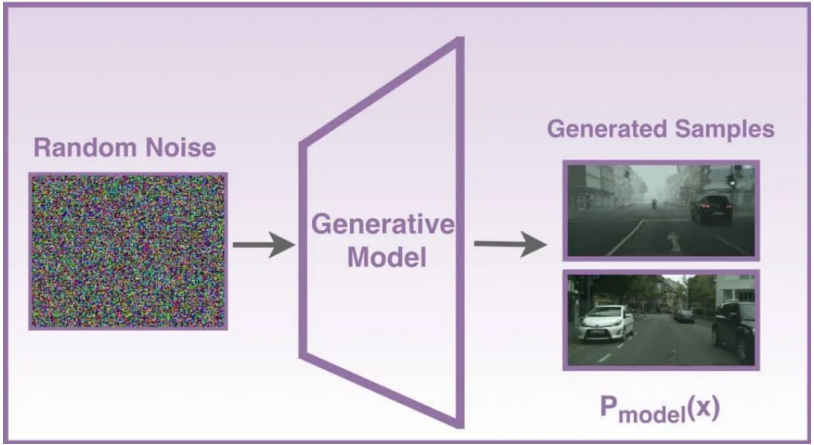


Figure: Image visualising the concept of diverse image generation from noise. (source: learnopencv.com)

Discriminative vs. Generative models

Discriminative Models

- **Goal:** Learn the conditional probability $P(y|\mathbf{x})$.

Discriminative vs. Generative models

Discriminative Models

- **Goal:** Learn the conditional probability $P(y|\mathbf{x})$.
- Learn a **decision boundary** between different classes.

Discriminative vs. Generative models

Discriminative Models

- **Goal:** Learn the conditional probability $P(y|\mathbf{x})$.
- Learn a **decision boundary** between different classes.
- **Asks:** Given this image $\mathbf{x}$, is it a cat or a dog?

Discriminative vs. Generative models

Discriminative Models

- **Goal:** Learn the conditional probability $P(y|\mathbf{x})$.
- Learn a **decision boundary** between different classes.
- **Asks:** Given this image $\mathbf{x}$, is it a cat or a dog?
- Examples: Classifiers, Regression models.

Discriminative vs. Generative models

Discriminative Models

- **Goal:** Learn the conditional probability $P(y|\mathbf{x})$.
- Learn a **decision boundary** between different classes.
- **Asks:** Given this image $\mathbf{x}$, is it a cat or a dog?
- Examples: Classifiers, Regression models.

Generative Models

- **Goal:** Learns the data distribution $P(\mathbf{x})$.

Discriminative vs. Generative models

Discriminative Models

- **Goal:** Learn the conditional probability $P(y|\mathbf{x})$.
- Learn a **decision boundary** between different classes.
- **Asks:** Given this image $\mathbf{x}$, is it a cat or a dog?
- Examples: Classifiers, Regression models.

Generative Models

- **Goal:** Learns the data distribution $P(\mathbf{x})$.
- Learn how the **data is generated**.

Discriminative vs. Generative models

Discriminative Models

- **Goal:** Learn the conditional probability $P(y|\mathbf{x})$.
- Learn a **decision boundary** between different classes.
- **Asks:** Given this image $\mathbf{x}$, is it a cat or a dog?
- Examples: Classifiers, Regression models.

Generative Models

- **Goal:** Learns the data distribution $P(\mathbf{x})$.
- Learn how the **data is generated**.
- **Asks:** What makes an image look like a cat?

Discriminative vs. Generative models

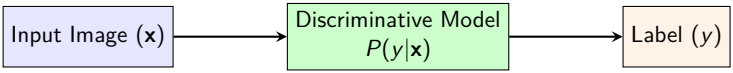
Discriminative Models

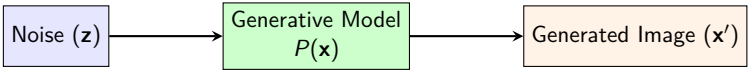
- **Goal:** Learn the conditional probability $P(y|\mathbf{x})$.
- Learn a **decision boundary** between different classes.
- **Asks:** Given this image $\mathbf{x}$, is it a cat or a dog?
- Examples: Classifiers, Regression models.

Generative Models

- **Goal:** Learns the data distribution $P(\mathbf{x})$.
- Learn how the **data is generated**.
- **Asks:** What makes an image look like a cat?
- Examples: Variational Autoencoders (VAEs), Generative Adversarial Networks (GANs).

Discriminative vs. Generative models





Discriminative vs. Generative models

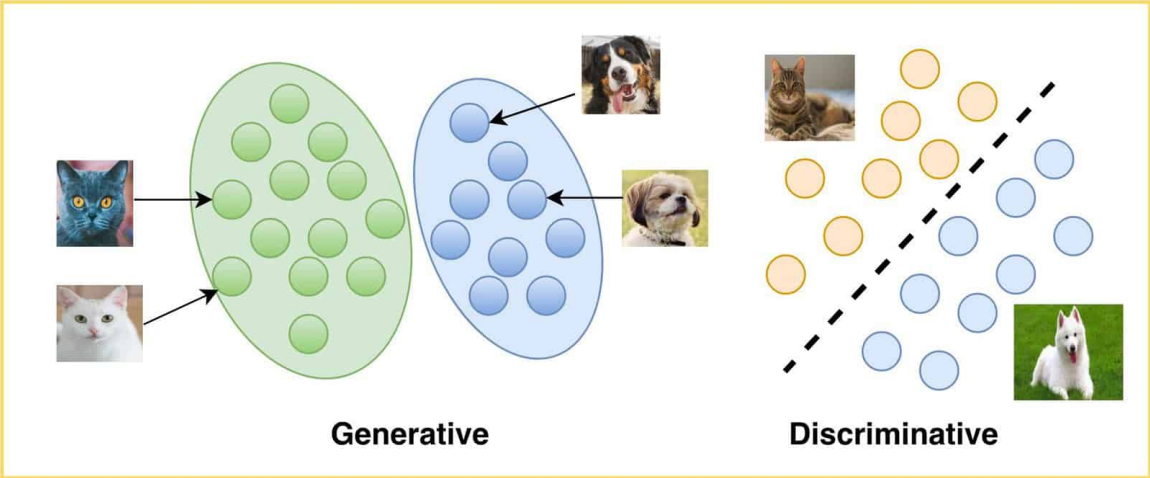


Figure: Different Learning patterns of Discriminative and Generative models.(source: learnopencv.com

Basics of Generative models

Latent Space, Noise Sampling and Types of Generative models

What is a Latent Space?

A **latent space** is a lower-dimensional representation that captures the essential, underlying structure of high-dimensional data.

What is a Latent Space?

A **latent space** is a lower-dimensional representation that captures the essential, underlying structure of high-dimensional data.

High-Dimensional Data

- Complex, noisy

What is a Latent Space?

A **latent space** is a lower-dimensional representation that captures the essential, underlying structure of high-dimensional data.

High-Dimensional Data

- Complex, noisy
- Redundant features

What is a Latent Space?

A **latent space** is a lower-dimensional representation that captures the essential, underlying structure of high-dimensional data.

High-Dimensional Data

- Complex, noisy
- Redundant features
- e.g., a 256x256 pixel image (65,536 dimensions)

What is a Latent Space?

A **latent space** is a lower-dimensional representation that captures the essential, underlying structure of high-dimensional data.

High-Dimensional Data

- Complex, noisy
- Redundant features
- e.g., a 256x256 pixel image (65,536 dimensions)

Latent Space

- Simplified, meaningful

What is a Latent Space?

A **latent space** is a lower-dimensional representation that captures the essential, underlying structure of high-dimensional data.

High-Dimensional Data

- Complex, noisy
- Redundant features
- e.g., a 256x256 pixel image (65,536 dimensions)

Latent Space

- Simplified, meaningful
- Captures core concepts

What is a Latent Space?

A **latent space** is a lower-dimensional representation that captures the essential, underlying structure of high-dimensional data.

High-Dimensional Data

- Complex, noisy
- Redundant features
- e.g., a 256x256 pixel image (65,536 dimensions)

Latent Space

- Simplified, meaningful
- Captures core concepts
- e.g., a 50-dimensional vector representing "age", "gender", "pose"

What is a Latent Space?

A **latent space** is a lower-dimensional representation that captures the essential, underlying structure of high-dimensional data.

High-Dimensional Data

- Complex, noisy
- Redundant features
- e.g., a 256x256 pixel image (65,536 dimensions)

Latent Space

- Simplified, meaningful
- Captures core concepts
- e.g., a 50-dimensional vector representing "age", "gender", "pose"

The primary goal was **dimensionality reduction** and **data interpretation**.

Example 1: Principal Component Analysis (PCA)

Core Idea: Find orthogonal axes (principal components) that capture the maximum variance in the data.

Example 1: Principal Component Analysis (PCA)

Core Idea: Find orthogonal axes (principal components) that capture the maximum variance in the data.

The Latent Space: The subspace spanned by the first k principal components.

Example 1: Principal Component Analysis (PCA)

Core Idea: Find orthogonal axes (principal components) that capture the maximum variance in the data.

The Latent Space: The subspace spanned by the first k principal components.

Application: Dimensionality reduction, data visualization. The "Eigenfaces" algorithm used PCA to represent faces in a compressed latent space.

Example 1: Principal Component Analysis (PCA)

Core Idea: Find orthogonal axes (principal components) that capture the maximum variance in the data.

The Latent Space: The subspace spanned by the first k principal components.

Application: Dimensionality reduction, data visualization. The "Eigenfaces" algorithm used PCA to represent faces in a compressed latent space.

Other Key Pre-Deep Learning Models

Factor Analysis (FA)

Idea: Assumes observed data is a linear combination of unobserved "factors" (latent variables) plus noise. Tries to find causal structure.
Application: Psychology, finance (e.g., finding "market sentiment").

Other Key Pre-Deep Learning Models

Factor Analysis (FA)

Idea: Assumes observed data is a linear combination of unobserved "factors" (latent variables) plus noise. Tries to find causal structure.
Application: Psychology, finance (e.g., finding "market sentiment").

Latent Dirichlet Allocation (LDA)

Idea: A generative model for text. Each document is a mix of topics, and each topic is a distribution over words.
The Latent Space: The space of topics.
Application: Topic modeling in large text corpora.

Other Key Pre-Deep Learning Models

Factor Analysis (FA)

Idea: Assumes observed data is a linear combination of unobserved "factors" (latent variables) plus noise. Tries to find causal structure.
Application: Psychology, finance (e.g., finding "market sentiment").

Latent Dirichlet Allocation (LDA)

Idea: A generative model for text. Each document is a mix of topics, and each topic is a distribution over words.
The Latent Space: The space of topics.
Application: Topic modeling in large text corpora.

Matrix Factorization (e.g., SVD)

Idea: Decomposes a large matrix (e.g., user-item ratings) into smaller matrices of latent factors.

Latent Space: Discovery & Inference

The primary goal was to **analyze existing data**.

Latent Space: Discovery & Inference

The primary goal was to **analyze existing data**.

The Process

Take a high-dimensional data point x and find its low-dimensional representation z in the latent space.

Latent Space: Discovery & Inference

The primary goal was to **analyze existing data**.

The Process

Take a high-dimensional data point x and find its low-dimensional representation z in the latent space.

$$\text{Data } (x) \longrightarrow \text{Encoder} \longrightarrow \text{Latent Space } (z)$$

Latent Space: Discovery & Inference

The primary goal was to **analyze existing data**.

The Process

Take a high-dimensional data point x and find its low-dimensional representation z in the latent space.

$$\text{Data } (x) \longrightarrow \text{Encoder} \longrightarrow \text{Latent Space } (z)$$

The Problem: The latent space was often unstructured. Picking a random point z and decoding it would usually produce garbage, not a valid new data point. The focus was on **inference**, not **generation**.

Latent Space: Sampling from Noise

The idea was to treat the latent space as a well-behaved **probability distribution** that we could sample from.

Latent Space: Sampling from Noise

The idea was to treat the latent space as a well-behaved **probability distribution** that we could sample from.

The New Process

Take a random vector of "noise" z from a simple distribution (e.g., Gaussian) and use a trained decoder to generate a new data point x.

Latent Space: Sampling from Noise

The idea was to treat the latent space as a well-behaved **probability distribution** that we could sample from.

The New Process

Take a random vector of "noise" z from a simple distribution (e.g., Gaussian) and use a trained decoder to generate a new data point x.

$$\text{Noise } (z) \longrightarrow \text{Decoder/Generator} \longrightarrow \text{New Data } (\hat{x})$$

Latent Space: Sampling from Noise

The idea was to treat the latent space as a well-behaved **probability distribution** that we could sample from.

The New Process

Take a random vector of "noise" z from a simple distribution (e.g., Gaussian) and use a trained decoder to generate a new data point x.

$$\text{Noise } (z) \longrightarrow \text{Decoder/Generator} \longrightarrow \text{New Data } (\hat{x})$$

This was enabled by deep learning models like **Variational Autoencoders (VAEs)** and **Generative Adversarial Networks (GANs)**, which learned to create a structured, continuous latent space.

How Noise Creates Variety: Toy vs. Deep Models

1. **Toy Example: Direct Probabilistic Sampling**
   - In the 2x2 grid, we assume we know the exact probability $P(\mathbf{x})$ for all 16 images.

How Noise Creates Variety: Toy vs. Deep Models

1. **Toy Example: Direct Probabilistic Sampling**
   - In the 2x2 grid, we assume we know the exact probability $P(\mathbf{x})$ for all 16 images.
   - To generate an image, we simply **sample directly from this learned discrete distribution**.

How Noise Creates Variety: Toy vs. Deep Models

1. **Toy Example: Direct Probabilistic Sampling**
   - In the 2x2 grid, we assume we know the exact probability $P(\mathbf{x})$ for all 16 images.
   - To generate an image, we simply **sample directly from this learned discrete distribution**.
   - If $P(\text{image A}) = 0.3$ and $P(\text{image B}) = 0.1$, Image A is 3 times more likely to be sampled.

How Noise Creates Variety: Toy vs. Deep Models

1. **Toy Example: Direct Probabilistic Sampling**
   - In the 2x2 grid, we assume we know the exact probability $P(\mathbf{x})$ for all 16 images.
   - To generate an image, we simply **sample directly from this learned discrete distribution**.
   - If $P(\text{image A}) = 0.3$ and $P(\text{image B}) = 0.1$, Image A is 3 times more likely to be sampled.
   - The "noise" comes from the randomness of this direct sampling process. Each sample is a random draw according to the learned probabilities. There's no separate "latent vector" input.

How Noise Creates Variety: Toy vs. Deep Models

2. Deep Generative Models (GANs, VAEs): Latent Space Sampling

- For high-dimensional data (like real images), we **cannot explicitly store** $P(\mathbf{x})$.

How Noise Creates Variety: Toy vs. Deep Models

2. Deep Generative Models (GANs, VAEs): Latent Space Sampling

- For high-dimensional data (like real images), we **cannot explicitly store** $P(\mathbf{x})$.
- Instead, these models learn a mapping function from a simple, low-dimensional "latent space" (often Gaussian noise) to the complex data space.

How Noise Creates Variety: Toy vs. Deep Models

2. Deep Generative Models (GANs, VAEs): Latent Space Sampling

- For high-dimensional data (like real images), we **cannot explicitly store** $P(\mathbf{x})$.
- Instead, these models learn a mapping function from a simple, low-dimensional "latent space" (often Gaussian noise) to the complex data space.
- The "noise vector" (z in GANs/VAEs) is a point sampled from this simple latent distribution.

How Noise Creates Variety: Toy vs. Deep Models

2. Deep Generative Models (GANs, VAEs): Latent Space Sampling

- For high-dimensional data (like real images), we **cannot explicitly store** $P(\mathbf{x})$.
- Instead, these models learn a mapping function from a simple, low-dimensional "latent space" (often Gaussian noise) to the complex data space.
- The "noise vector" (z in GANs/VAEs) is a point sampled from this simple latent distribution.
- The Generator/Decoder then transforms this noise vector into a coherent, high-dimensional output (e.g., an image).

How Noise Creates Variety: Toy vs. Deep Models

2. Deep Generative Models (GANs, VAEs): Latent Space Sampling

- For high-dimensional data (like real images), we **cannot explicitly store** $P(\mathbf{x})$.
- Instead, these models learn a mapping function from a simple, low-dimensional "latent space" (often Gaussian noise) to the complex data space.
- The "noise vector" (z in GANs/VAEs) is a point sampled from this simple latent distribution.
- The Generator/Decoder then transforms this noise vector into a coherent, high-dimensional output (e.g., an image).
- The "noise" is the source of variety. Different z vectors map to different generated outputs, allowing the model to produce diverse samples from the learned data manifold.

How Noise Creates Variety: Toy vs. Deep Models

Analogy

- **Toy Model:** Rolling a loaded die (the die's faces are the 16 images, the loading is $P(\mathbf{x})$). The randomness is in the roll.

How Noise Creates Variety: Toy vs. Deep Models

Analogy

- **Toy Model:** Rolling a loaded die (the die's faces are the 16 images, the loading is $P(\mathbf{x})$). The randomness is in the roll.
- **Deep Model:** You pick a random set of DNA instructions (z), and then an organism grows from it (the Generator). Different DNA leads to different organisms.

Beyond Toy Grids: Real-world Generative Models

- For complex data like high-resolution images, text, or audio, explicit probability distributions are intractable.

Beyond Toy Grids: Real-world Generative Models

- For complex data like high-resolution images, text, or audio, explicit probability distributions are intractable.
- Modern generative models use neural networks to learn complex, high-dimensional distributions.

Beyond Toy Grids: Real-world Generative Models

- For complex data like high-resolution images, text, or audio, explicit probability distributions are intractable.
- Modern generative models use neural networks to learn complex, high-dimensional distributions.
- Key types include:
  - Variational Autoencoders (VAEs)
  - Generative Adversarial Networks (GANs)
  - Autoregressive Models (e.g., PixelRNN, WaveNet)
  - Flow-based Models
  - Diffusion Models (currently state-of-the-art for image generation)

Enumeration to Latent Noise

Pre-Deep Learning

- Focused on explicitly modeling $P(\mathbf{x})$.

Enumeration to Latent Noise

Pre-Deep Learning

- Focused on explicitly modeling $P(\mathbf{x})$.
- For simple data , this meant enumerating probabilities.

Enumeration to Latent Noise

Pre-Deep Learning

- Focused on explicitly modeling $P(\mathbf{x})$.
- For simple data , this meant enumerating probabilities.
- For slightly more complex data, people used simpler parametric distributions (e.g., Gaussian Mixture Models) or graphical models (e.g., Bayesian Networks, Markov Random Fields).

Enumeration to Latent Noise

Pre-Deep Learning

- Focused on explicitly modeling $P(\mathbf{x})$.
- For simple data , this meant enumerating probabilities.
- For slightly more complex data, people used simpler parametric distributions (e.g., Gaussian Mixture Models) or graphical models (e.g., Bayesian Networks, Markov Random Fields).
- **Limitation:** These models struggled immensely as data dimensionality increased. Defining good explicit distributions for high-dimensional, highly structured data (like images) was almost impossible.

Enumeration to Latent Noise

Pre-Deep Learning

- Focused on explicitly modeling $P(\mathbf{x})$.
- For simple data , this meant enumerating probabilities.
- For slightly more complex data, people used simpler parametric distributions (e.g., Gaussian Mixture Models) or graphical models (e.g., Bayesian Networks, Markov Random Fields).
- **Limitation:** These models struggled immensely as data dimensionality increased. Defining good explicit distributions for high-dimensional, highly structured data (like images) was almost impossible.

The Need for Latent Space / Noise Input:

- Real images (e.g., a cat) exist on a very low-dimensional "manifold" within the vast pixel space. Most random pixel configurations are just noise.

Enumeration to Latent Noise

Pre-Deep Learning

- Focused on explicitly modeling $P(\mathbf{x})$.
- For simple data , this meant enumerating probabilities.
- For slightly more complex data, people used simpler parametric distributions (e.g., Gaussian Mixture Models) or graphical models (e.g., Bayesian Networks, Markov Random Fields).
- **Limitation:** These models struggled immensely as data dimensionality increased. Defining good explicit distributions for high-dimensional, highly structured data (like images) was almost impossible.

The Need for Latent Space / Noise Input:

- Real images (e.g., a cat) exist on a very low-dimensional "manifold" within the vast pixel space. Most random pixel configurations are just noise.
- **Problem:** How do we randomly "find" a point on this manifold without explicitly mapping out the entire high-dimensional probability distribution?

Enumeration to Latent Noise

Pre-Deep Learning

- Focused on explicitly modeling $P(\mathbf{x})$.
- For simple data , this meant enumerating probabilities.
- For slightly more complex data, people used simpler parametric distributions (e.g., Gaussian Mixture Models) or graphical models (e.g., Bayesian Networks, Markov Random Fields).
- **Limitation:** These models struggled immensely as data dimensionality increased. Defining good explicit distributions for high-dimensional, highly structured data (like images) was almost impossible.

The Need for Latent Space / Noise Input:

- Real images (e.g., a cat) exist on a very low-dimensional "manifold" within the vast pixel space. Most random pixel configurations are just noise.
- **Problem:** How do we randomly "find" a point on this manifold without explicitly mapping out the entire high-dimensional probability distribution?
- **Solution:** Instead of trying to directly sample from $P(\mathbf{x})$ in pixel space, let's learn a *mapping* from a simpler, continuous distribution (the latent noise z) to the data manifold.

Intuition for Latent Noise

The Intuition

1. Imagine the "space of all possible images." Only a tiny fraction are meaningful.

Intuition for Latent Noise

The Intuition

1. Imagine the "space of all possible images." Only a tiny fraction are meaningful.
2. We can't pick random pixels and get a cat.

Intuition for Latent Noise

The Intuition

1. Imagine the "space of all possible images." Only a tiny fraction are meaningful.
2. We can't pick random pixels and get a cat.
3. But we can pick a random point in a simpler, smaller "idea space" (the latent space Z).

Intuition for Latent Noise

The Intuition

1. Imagine the "space of all possible images." Only a tiny fraction are meaningful.
2. We can't pick random pixels and get a cat.
3. But we can pick a random point in a simpler, smaller "idea space" (the latent space Z).
4. We then train a complex function (the Generator/Decoder) to transform that random "idea" into a corresponding meaningful image.

Intuition for Latent Noise

The Intuition

1. Imagine the "space of all possible images." Only a tiny fraction are meaningful.
2. We can't pick random pixels and get a cat.
3. But we can pick a random point in a simpler, smaller "idea space" (the latent space Z).
4. We then train a complex function (the Generator/Decoder) to transform that random "idea" into a corresponding meaningful image.
5. Each unique z vector acts as a unique "seed" or "recipe" for a specific image, ensuring variety while staying on the data manifold.

Intuition for Latent Noise

The Transition

- This concept was formalized with models like Autoencoders (AEs) for learning representations, then extended to Variational Autoencoders (VAEs) in 2013.

Intuition for Latent Noise

The Transition

- This concept was formalized with models like Autoencoders (AEs) for learning representations, then extended to Variational Autoencoders (VAEs) in 2013.
- Generative Adversarial Networks (GANs) in 2014 provided an alternative way to learn this mapping without directly modeling $p(\mathbf{x}|z)$.

Intuition for Latent Noise

The Transition

- This concept was formalized with models like Autoencoders (AEs) for learning representations, then extended to Variational Autoencoders (VAEs) in 2013.
- Generative Adversarial Networks (GANs) in 2014 provided an alternative way to learn this mapping without directly modeling $p(\mathbf{x}|z)$.
- Diffusion Models emerged more recently as another powerful approach using structured noise.

Summary of the Transition

Pre-Deep Learning (e.g., PCA)	**Modern Generative (e.g., GANs)**
Goal: Discovery & Representation	**Goal:** Generation & Creation

Summary of the Transition

Pre-Deep Learning (e.g., PCA)	Modern Generative (e.g., GANs)
Goal: Discovery & Representation	**Goal:** Generation & Creation
Latent Space: Derived *from* data. Often disjointed.	**Latent Space:** Has an *imposed* structure (e.g., Gaussian). Continuous.

Summary of the Transition

Pre-Deep Learning (e.g., PCA)	Modern Generative (e.g., GANs)
Goal: Discovery & Representation	**Goal:** Generation & Creation
Latent Space: Derived *from* data. Often disjointed.	**Latent Space:** Has an *imposed* structure (e.g., Gaussian). Continuous.
Direction: Data (x) $\rightarrow$ Latent (z)	**Direction:** Noise (z) $\rightarrow$ Data ($\hat{x}$)

Summary of the Transition

Pre-Deep Learning (e.g., PCA)	**Modern Generative (e.g., GANs)**
Goal: Discovery & Representation	**Goal:** Generation & Creation
Latent Space: Derived *from* data. Often disjointed.	**Latent Space:** Has an *imposed* structure (e.g., Gaussian). Continuous.
Direction: Data (x) $\rightarrow$ Latent (z)	**Direction:** Noise (z) $\rightarrow$ Data ($\hat{x}$)
Question: "What are the hidden factors in my data?"	**Question:** "How can I create new data that looks like my data?"

Table: The paradigm shift from data analysis to data synthesis.

How to Classify Generative Models?

Generative models learn the underlying probability distribution of the data, $p(x)$. They can be categorized based on **how they model this distribution**.

How to Classify Generative Models?

Generative models learn the underlying probability distribution of the data, $p(x)$. They can be categorized based on **how they model this distribution**.

1. Explicit Density Models

They explicitly define and learn a function for $p(x)$. You can ask "what is the probability of this image?"

How to Classify Generative Models?

Generative models learn the underlying probability distribution of the data, $p(x)$. They can be categorized based on **how they model this distribution**.

1. Explicit Density Models

They explicitly define and learn a function for $p(x)$. You can ask "what is the probability of this image?"

2. Implicit Density Models

They learn a process to *sample* from $p(x)$ without ever defining the function itself.

1. Explicit Density Models

These models allow for direct evaluation of the data likelihood, $p(x)$.

1. Explicit Density Models

These models allow for direct evaluation of the data likelihood, $p(x)$.

a) Tractable Density

The density $p(x)$ can be computed exactly.

1. Explicit Density Models

These models allow for direct evaluation of the data likelihood, $p(x)$.

a) Tractable Density

The density $p(x)$ can be computed exactly.

- **Autoregressive Models:** Model joint distribution as a product of conditionals. Generate pixel-by-pixel. (*Examples: PixelRNN, PixelCNN, GPT-2*)

1. Explicit Density Models

These models allow for direct evaluation of the data likelihood, $p(x)$.

a) Tractable Density

The density $p(x)$ can be computed exactly.

- **Autoregressive Models:** Model joint distribution as a product of conditionals. Generate pixel-by-pixel. (*Examples: PixelRNN, PixelCNN, GPT-2*)
- **Normalizing Flows:** Transform a simple distribution into a complex one via invertible functions. (*Examples: RealNVP, Glow*)

1. Explicit Density Models

These models allow for direct evaluation of the data likelihood, $p(x)$.

a) Tractable Density

The density $p(x)$ can be computed exactly.

- **Autoregressive Models:** Model joint distribution as a product of conditionals. Generate pixel-by-pixel. (*Examples: PixelRNN, PixelCNN, GPT-2*)
- **Normalizing Flows:** Transform a simple distribution into a complex one via invertible functions. (*Examples: RealNVP, Glow*)

b) Approximate Density

The density $p(x)$ is intractable, so we optimize an approximation.

1. Explicit Density Models

These models allow for direct evaluation of the data likelihood, $p(x)$.

a) Tractable Density

The density $p(x)$ can be computed exactly.

- **Autoregressive Models:** Model joint distribution as a product of conditionals. Generate pixel-by-pixel. (*Examples: PixelRNN, PixelCNN, GPT-2*)
- **Normalizing Flows:** Transform a simple distribution into a complex one via invertible functions. (*Examples: RealNVP, Glow*)

b) Approximate Density

The density $p(x)$ is intractable, so we optimize an approximation.

- **Variational Autoencoders (VAEs):** Maximize a lower bound on the likelihood (ELBO). (*Examples: VAE, VQ-VAE*)

1. Explicit Density Models

These models allow for direct evaluation of the data likelihood, $p(x)$.

a) Tractable Density

The density $p(x)$ can be computed exactly.

- **Autoregressive Models:** Model joint distribution as a product of conditionals. Generate pixel-by-pixel. (*Examples: PixelRNN, PixelCNN, GPT-2*)
- **Normalizing Flows:** Transform a simple distribution into a complex one via invertible functions. (*Examples: RealNVP, Glow*)

b) Approximate Density

The density $p(x)$ is intractable, so we optimize an approximation.

- **Variational Autoencoders (VAEs):** Maximize a lower bound on the likelihood (ELBO). (*Examples: VAE, VQ-VAE*)
- **Diffusion / Score-Based Models:** Systematically add noise to data, then learn to reverse the process. State-of-the-art for images. (*Examples: DDPM, Stable Diffusion, DALL-E 2*)

2. Implicit Density Models

These models learn to generate data without ever defining the probability function $p(x)$.

2. Implicit Density Models

These models learn to generate data without ever defining the probability function $p(x)$.

Generative Adversarial Networks (GANs)

The dominant family in this category.

2. Implicit Density Models

These models learn to generate data without ever defining the probability function $p(x)$.

Generative Adversarial Networks (GANs)

The dominant family in this category.

- A two-player game between a **Generator** and a **Discriminator**.

2. Implicit Density Models

These models learn to generate data without ever defining the probability function $p(x)$.

Generative Adversarial Networks (GANs)

The dominant family in this category.

- A two-player game between a **Generator** and a **Discriminator**.
- **Generator:** Creates fake data from random noise. Tries to fool the discriminator.

2. Implicit Density Models

These models learn to generate data without ever defining the probability function $p(x)$.

Generative Adversarial Networks (GANs)

The dominant family in this category.

- A two-player game between a **Generator** and a **Discriminator**.
- **Generator:** Creates fake data from random noise. Tries to fool the discriminator.
- **Discriminator:** Tries to distinguish real data from fake data.
- Through this adversarial process, the generator learns the true data distribution implicitly.

Examples: DCGAN, StyleGAN, CycleGAN

Taxonomy Summary Table

Category	Sub-Category	Key Idea	Examples
Explicit Density	**Tractable**	Directly compute $p(x)$	PixelCNN, Glow
	Approximate	Optimize a proxy for $p(x)$	VAEs, Diffusion Models
Implicit Density	-	Learn to sample from $p(x)$	GANs, StyleGAN

- **GANs** are known for sharp, high-fidelity samples but can be unstable to train.
- **VAEs** are stable and have a well-behaved latent space but can produce blurry results.
- **Flows** offer exact likelihoods but can be computationally expensive.
- **Diffusion Models** are the current leaders in sample quality and diversity but are slow to sample.

Conclusion

- The concept of a **latent space** has been fundamental in machine learning for decades, originally for data analysis and discovery.

Conclusion

- The concept of a **latent space** has been fundamental in machine learning for decades, originally for data analysis and discovery.
- The modern generative era began with a conceptual shift: using **random noise** as an input to sample from a learned, structured latent space.

Conclusion

- The concept of a **latent space** has been fundamental in machine learning for decades, originally for data analysis and discovery.
- The modern generative era began with a conceptual shift: using **random noise** as an input to sample from a learned, structured latent space.
- Today's powerful generative models (GANs, VAEs, Diffusion) are the direct descendants of this idea, each with a unique approach to learning the underlying data distribution.

Thank you!

Introduction to GANs

The GAN architecture, Training and Challenges

Intuition behind GANs

Proposed by Ian Goodfellow et al. in 2014, the core idea is simple in hindsight: **make neural networks compete against each other in the hope that this competition will push them to excel**.

Intuition behind GANs

Proposed by Ian Goodfellow et al. in 2014, the core idea is simple in hindsight: **make neural networks compete against each other in the hope that this competition will push them to excel**.

Think of it as a game between a **Artist** and a **Critic**.

Intuition behind GANs

Proposed by Ian Goodfellow et al. in 2014, the core idea is simple in hindsight: **make neural networks compete against each other in the hope that this competition will push them to excel**.

Think of it as a game between a **Artist** and a **Critic**.

The Aspiring Artist (Generator)

Tries to create paintings in the style of a master (e.g., Picasso). The artist has **never seen a real Picasso**, but learns by trying to fool the art critic.

Intuition behind GANs

Proposed by Ian Goodfellow et al. in 2014, the core idea is simple in hindsight: **make neural networks compete against each other in the hope that this competition will push them to excel**.

Think of it as a game between a **Artist** and a **Critic**.

The Aspiring Artist (Generator)

Tries to create paintings in the style of a master (e.g., Picasso). The artist has **never seen a real Picasso**, but learns by trying to fool the art critic.

The Art Critic (Discriminator)

Tries to distinguish between genuine Picasso paintings and the artist's forgeries. The critic learns by studying both the **real masterpieces and the fakes**.

Intuition behind GANs

Proposed by Ian Goodfellow et al. in 2014, the core idea is simple in hindsight: **make neural networks compete against each other in the hope that this competition will push them to excel**.

Think of it as a game between a **Artist** and a **Critic**.

The Aspiring Artist (Generator)

Tries to create paintings in the style of a master (e.g., Picasso). The artist has **never seen a real Picasso**, but learns by trying to fool the art critic.

The Art Critic (Discriminator)

Tries to distinguish between genuine Picasso paintings and the artist's forgeries. The critic learns by studying both the **real masterpieces and the fakes**.

The Creative Competition

The game ends when the artist's forgeries are so good that the critic is fooled about half the time.

The GAN architecture

A GAN is composed of two neural networks: the **Generator** and the **Discriminator**.

The GAN architecture

A GAN is composed of two neural networks: the **Generator** and the **Discriminator**.

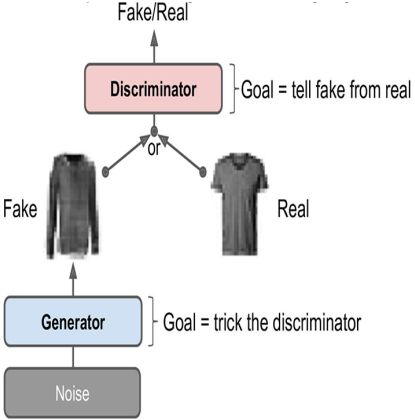


Figure: GAN architecture.

Source: Hands-On Machine Learning with Scikit-Learn, Keras TensorFlow.

The Models and their Goals

1. The Discriminator (D): The Classifier

The Discriminator's goal is to become an expert at distinguishing real data from fake data. It is trained to maximize the probability of making a correct classification.

The Models and their Goals

1. The Discriminator (D): The Classifier

The Discriminator's goal is to become an expert at distinguishing real data from fake data. It is trained to maximize the probability of making a correct classification.

Its objective is to **maximize** this function:

$$\max_{D} \left(\underbrace{\mathbb{E}_{\mathbf{x}\sim p_{data}(\mathbf{x})}[\log D(\mathbf{x})]}_{\text{Correctly label real as real}} + \underbrace{\mathbb{E}_{\mathbf{z}\sim p_{\mathbf{z}}(\mathbf{z})}[\log(1 - D(G(\mathbf{z})))]}_{\text{Correctly label fake as fake}} \right)$$

The Models and their Goals

2. The Generator (G): The Creator

The Generator's goal is to create data so realistic that the Discriminator cannot tell it's fake. It wants to fool the Discriminator.

The Models and their Goals

2. The Generator (G): The Creator

The Generator's goal is to create data so realistic that the Discriminator cannot tell it's fake. It wants to fool the Discriminator.

Its objective is to **minimize** the second term of the Discriminator's objective:

$$\min_G \left(\mathbb{E}_{\mathbf{z} \sim p_{\mathbf{z}}(\mathbf{z})} [\log(1 - D(G(\mathbf{z})))] \right)$$

This is equivalent to training G to maximize $\log(D(G(\mathbf{z})))$, pushing the Discriminator's output for fake images towards 1.

GAN Training Objective

Now, we combine these two competing objectives into a single formula for Loss Function.

GAN Training Objective

Now, we combine these two competing objectives into a single formula for Loss Function.

The Combined Minimax Objective

The full training objective pits the Generator against the Discriminator. The Generator tries to minimize the value function that the Discriminator tries to maximize:

$$\min_G \max_D V(D, G) = \mathbb{E}_{\mathbf{x} \sim p_{data}(\mathbf{x})}[\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{z} \sim p_{\mathbf{z}}(\mathbf{z})}[\log(1 - D(G(\mathbf{z})))]$$

GAN Training Objective

Now, we combine these two competing objectives into a single formula for Loss Function.

The Combined Minimax Objective

The full training objective pits the Generator against the Discriminator. The Generator tries to minimize the value function that the Discriminator tries to maximize:

$$\min_G \max_D V(D, G) = \mathbb{E}_{\mathbf{x} \sim p_{data}(\mathbf{x})}[\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{z} \sim p_{\mathbf{z}}(\mathbf{z})}[\log(1 - D(G(\mathbf{z})))]$$

Because the two networks are locked in a direct competition where one's success is the other's failure, this method is called **Adversarial Learning**

GAN Training Objective

Now, we combine these two competing objectives into a single formula for Loss Function.

The Combined Minimax Objective

The full training objective pits the Generator against the Discriminator. The Generator tries to minimize the value function that the Discriminator tries to maximize:

$$\min_G \max_D V(D, G) = \mathbb{E}_{\mathbf{x} \sim p_{data}(\mathbf{x})}[\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{z} \sim p_{\mathbf{z}}(\mathbf{z})}[\log(1 - D(G(\mathbf{z})))]$$

Because the two networks are locked in a direct competition where one's success is the other's failure, this method is called **Adversarial Learning**

The Goal: Nash Equilibrium

The ideal outcome of this game is to find a **Nash Equilibrium**. At this point, the Generator produces perfectly realistic data, and the Discriminator is unable to distinguish real from fake, outputting a 50% probability for all inputs.

GAN Training Algorithm: Phase 1 - Train the Discriminator

Objective: Teach D to distinguish real from fake

1. **Sample data:**
   - Real images: $\mathbf{x} \sim p_{\text{data}}$

from dataset

Real $\mathbf{x}$

GAN Training Algorithm: Phase 1 - Train the Discriminator

Objective: Teach D to distinguish real from fake

1. **Sample data:**
   - Real images: $\mathbf{x} \sim p_{\text{data}}$
   - Noise vectors: $\mathbf{z} \sim p_{\mathbf{z}}$

from dataset

Real $\mathbf{x}$

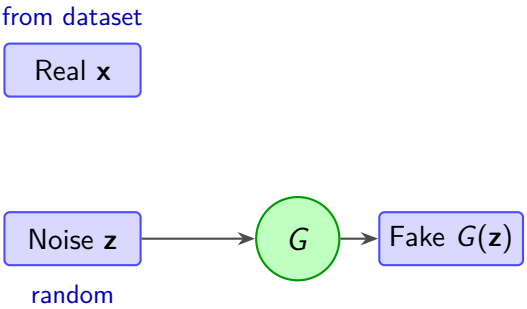
Noise $\mathbf{z}$

random

GAN Training Algorithm: Phase 1 - Train the Discriminator

Objective: Teach D to distinguish real from fake

1. **Sample data:**
   - Real images: $\mathbf{x} \sim p_{\text{data}}$
   - Noise vectors: $\mathbf{z} \sim p_{\mathbf{z}}$
   - Generate fakes: $G(\mathbf{z})$

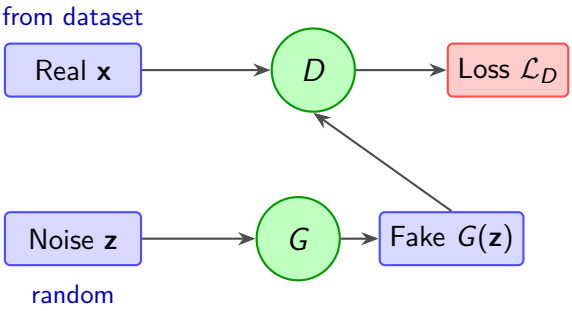


GAN Training Algorithm: Phase 1 - Train the Discriminator

Objective: Teach D to distinguish real from fake

1. **Sample data:**
   - Real images: $\mathbf{x} \sim p_{\text{data}}$
   - Noise vectors: $\mathbf{z} \sim p_{\mathbf{z}}$
   - Generate fakes: $G(\mathbf{z})$
2. **Maximize** discriminator objective:

$$\max_{\theta_d} \mathbb{E}_{\mathbf{x}}[\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{z}}[\log(1 - D(G(\mathbf{z})))]$$



GAN Training Algorithm: Phase 1 - Train the Discriminator

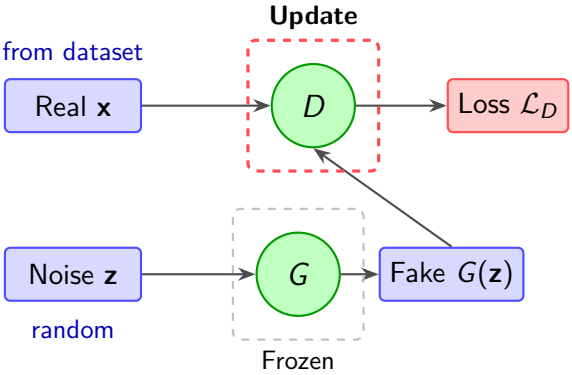
Objective: Teach D to distinguish real from fake

1. **Sample data:**
   - Real images: $\mathbf{x} \sim p_{\text{data}}$
   - Noise vectors: $\mathbf{z} \sim p_{\mathbf{z}}$
   - Generate fakes: $G(\mathbf{z})$
2. **Maximize** discriminator objective:

$$\max_{\theta_d} \mathbb{E}_{\mathbf{x}}[\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{z}}[\log(1 - D(G(\mathbf{z})))]$$

3. **Gradient ascent** step (with generator freezed):

$$\theta_d \leftarrow \theta_d + \alpha \nabla_{\theta_d} \frac{1}{m} \sum_{i=1}^{m} \left[\log D(\mathbf{x}^{(i)}) + \log(1 - D(G(\mathbf{z}^{(i)}))) \right]$$



Result: D improves at detection

GAN Training Algorithm: Phase 2 - Train the Generator

Objective: Teach G to fool the discriminator

1. **Sample new noise:**
   - $\mathbf{z} \sim p_{\mathbf{z}}$

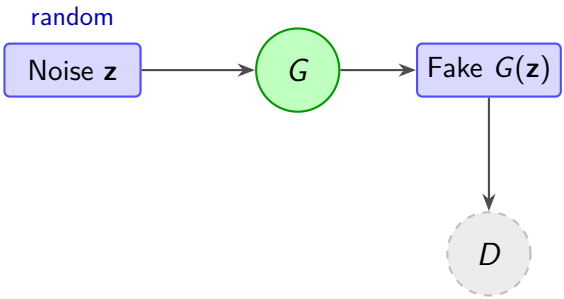
random

Noise $\mathbf{z}$

GAN Training Algorithm: Phase 2 - Train the Generator

Objective: Teach G to fool the discriminator

1. **Sample new noise:**
   - $\mathbf{z} \sim p_{\mathbf{z}}$
   - Generate: $G(\mathbf{z})$

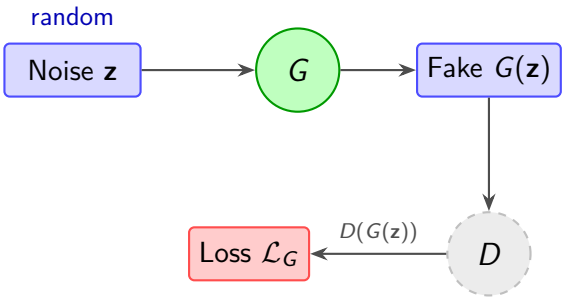


GAN Training Algorithm: Phase 2 - Train the Generator

Objective: Teach G to fool the discriminator

1. **Sample new noise:**
   - $\mathbf{z} \sim p_{\mathbf{z}}$
   - Generate: $G(\mathbf{z})$
2. **Minimize** generator loss (or maximize D confusion):

$$\min_{\theta_g} \mathbb{E}_{\mathbf{z}}[\log(1 - D(G(\mathbf{z})))]$$



GAN Training Algorithm: Phase 2 - Train the Generator

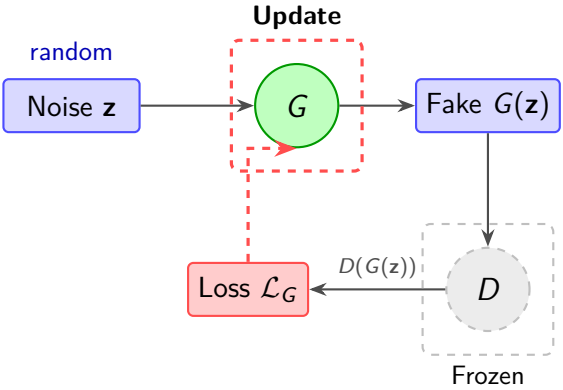
Objective: Teach G to fool the discriminator

1. **Sample new noise:**
   - $\mathbf{z} \sim p_{\mathbf{z}}$
   - Generate: $G(\mathbf{z})$
2. **Minimize** generator loss (or maximize D confusion):

$$\min_{\theta_g} \mathbb{E}_{\mathbf{z}}[\log(1 - D(G(\mathbf{z})))]$$

3. **Gradient descent** step (with frozen discriminator):

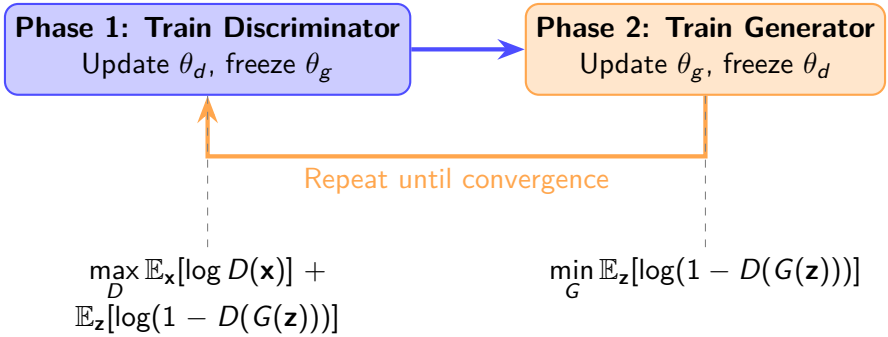
$$\theta_g \leftarrow \theta_g - \alpha \nabla_{\theta_g} \frac{1}{m} \sum_{i=1}^{m} \log(1 - D(G(\mathbf{z}^{(i)})))$$



Result: G improves at generation

(Gradients flow back through D to update G)

GAN Training: Alternating Optimization



Minimax Game: $\min_G \max_D V(D, G) = \mathbb{E}_{\mathbf{x}}[\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{z}}[\log(1 - D(G(\mathbf{z})))]$

DCGAN: Deep Convolutional GAN

DCGANs were a major step forward, successfully using deep convolutional nets for larger images. They proposed several guidelines for stable training:

DCGAN: Deep Convolutional GAN

DCGANs were a major step forward, successfully using deep convolutional nets for larger images. They proposed several guidelines for stable training:

- **Replace Pooling Layers:** Use strided convolutions in the discriminator (for downsampling) and transposed convolutions in the generator (for upsampling).

DCGAN: Deep Convolutional GAN

DCGANs were a major step forward, successfully using deep convolutional nets for larger images. They proposed several guidelines for stable training:

- **Replace Pooling Layers:** Use strided convolutions in the discriminator (for downsampling) and transposed convolutions in the generator (for upsampling).
- **Use Batch Normalization:** Apply BatchNorm in both networks, but not on the generator's output or the discriminator's input layer.

DCGAN: Deep Convolutional GAN

DCGANs were a major step forward, successfully using deep convolutional nets for larger images. They proposed several guidelines for stable training:

- **Replace Pooling Layers:** Use strided convolutions in the discriminator (for downsampling) and transposed convolutions in the generator (for upsampling).
- **Use Batch Normalization:** Apply BatchNorm in both networks, but not on the generator's output or the discriminator's input layer.
- **Remove Fully Connected Layers:** For deeper architectures, remove fully connected hidden layers.

DCGAN: Deep Convolutional GAN

DCGANs were a major step forward, successfully using deep convolutional nets for larger images. They proposed several guidelines for stable training:

- **Replace Pooling Layers:** Use strided convolutions in the discriminator (for downsampling) and transposed convolutions in the generator (for upsampling).
- **Use Batch Normalization:** Apply BatchNorm in both networks, but not on the generator's output or the discriminator's input layer.
- **Remove Fully Connected Layers:** For deeper architectures, remove fully connected hidden layers.
- **Activation Functions:**
  - Use **ReLU** in the generator for all layers except the output, which should use **Tanh**.
  - Use **LeakyReLU** in the discriminator for all layers.

DCGAN generations

Figure: New face generation from DCGAN

DCGAN generations

Figure: New face generation from DCGAN

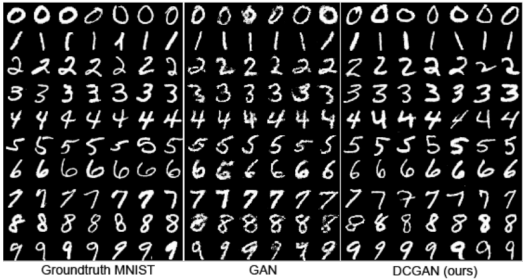


Figure: Qualitative Comparison between GAN and DCGAN

(source: "UNSUPERVISED REPRESENTATION LEARNING WITH DEEP CONVOLUTIONAL GENERATIVE ADVERSARIAL NETWORKS)

StyleGANs

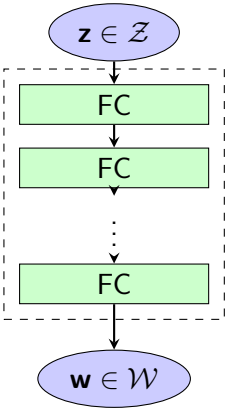
A state-of-the-art architecture from Nvidia that significantly improved image quality by incorporating **style transfer** techniques. The generator is composed of two main networks:

StyleGANs

A state-of-the-art architecture from Nvidia that significantly improved image quality by incorporating **style transfer** techniques. The generator is composed of two main networks:

1. Mapping Network

- A multi-layer MLP that maps the input latent code $\mathbf{z}$ into an intermediate latent space $\mathbf{w}$.

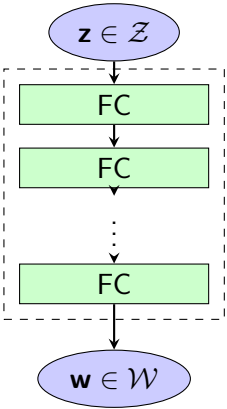


StyleGANs

A state-of-the-art architecture from Nvidia that significantly improved image quality by incorporating **style transfer** techniques. The generator is composed of two main networks:

1. Mapping Network

- A multi-layer MLP that maps the input latent code $\mathbf{z}$ into an intermediate latent space $\mathbf{w}$.
- This disentangles the latent space and reduces correlations.

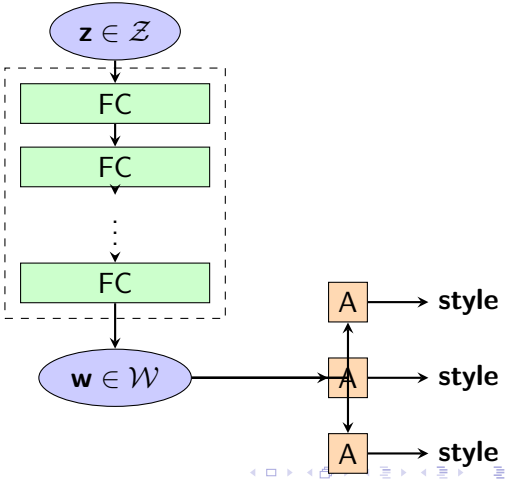


StyleGANs

A state-of-the-art architecture from Nvidia that significantly improved image quality by incorporating **style transfer** techniques. The generator is composed of two main networks:

1. Mapping Network

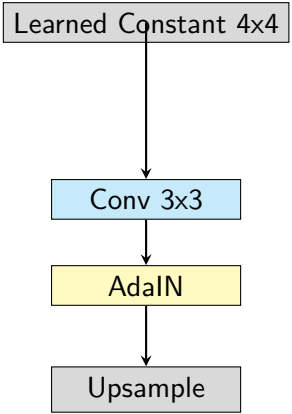
- A multi-layer MLP that maps the input latent code **z** into an intermediate latent space **w**.
- This disentangles the latent space and reduces correlations.
- From **w**, multiple **style vectors** are produced via affine transformations. These control different visual features (e.g., hair color, face shape) at different scales.



StyleGANs

2. Synthesis Network

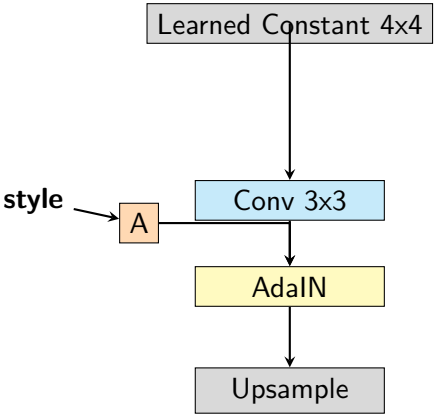
- Responsible for generating the image, starting from a learned constant input.



StyleGANs

2. Synthesis Network

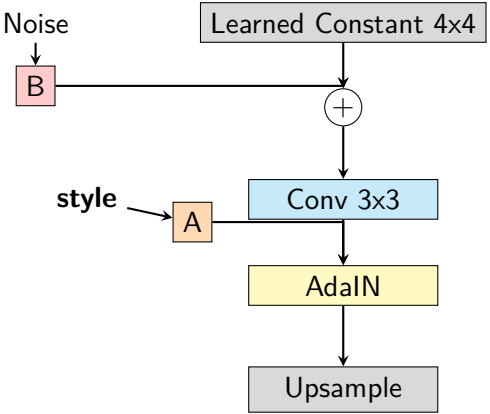
- Responsible for generating the image, starting from a learned constant input.
- **AdaIN (Adaptive Instance Normalization):** Uses the style vectors from the mapping network to control the scale and bias of features at each convolutional layer.



StyleGANs

2. Synthesis Network

- Responsible for generating the image, starting from a learned constant input.
- **AdaIN (Adaptive Instance Normalization):** Uses the style vectors from the mapping network to control the scale and bias of features at each convolutional layer.
- **Stochastic Variation:** Adds explicit noise at different layers to model random features like freckles or hair placement, which is more efficient than encoding this in the latent code **z**.



StyleGAN Architecture

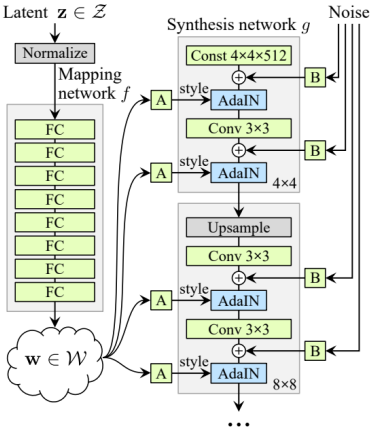


Figure: StyleGAN generator Architecture.
(source: "A Style-Based Generator Architecture for Generative Adversarial Networks")

StyleGAN Illustration

Figure: StyleGAN Face Generation.
(source: "A Style-Based Generator Architecture for Generative Adversarial Networks")

Training challenges in GAN

Training GANs can be notoriously difficult and unstable.

Training challenges in GAN

Training GANs can be notoriously difficult and unstable.

Mode Collapse

This is the biggest difficulty, where the generator's outputs become less diverse.

Training challenges in GAN

Training GANs can be notoriously difficult and unstable.

Mode Collapse

This is the biggest difficulty, where the generator's outputs become less diverse.

- **How it happens:** The generator might find one type of output (e.g., images of shoes) that is particularly good at fooling the discriminator.

Training challenges in GAN

Training GANs can be notoriously difficult and unstable.

Mode Collapse

This is the biggest difficulty, where the generator's outputs become less diverse.

- **How it happens:** The generator might find one type of output (e.g., images of shoes) that is particularly good at fooling the discriminator.
- It then focuses on producing only that output, forgetting how to generate other classes.

Training challenges in GAN

Training GANs can be notoriously difficult and unstable.

Mode Collapse

This is the biggest difficulty, where the generator's outputs become less diverse.

- **How it happens:** The generator might find one type of output (e.g., images of shoes) that is particularly good at fooling the discriminator.
- It then focuses on producing only that output, forgetting how to generate other classes.
- The GAN may cycle through a few modes, never learning the full data distribution.

Mode Collapse Illustration

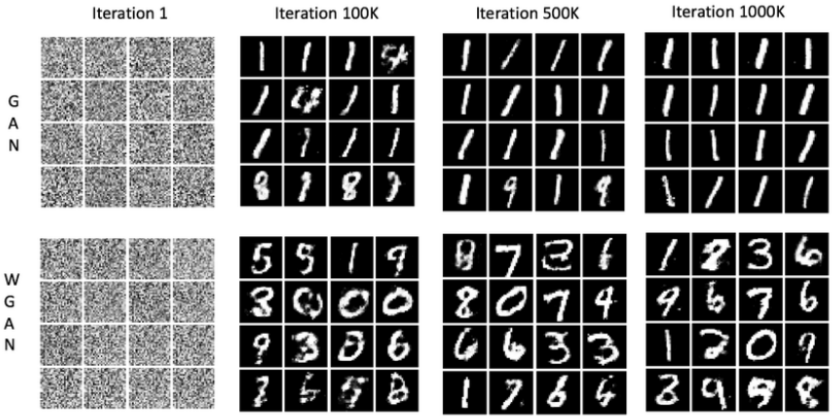


Figure: Illustration of Mode Collapse in Traditional GAN, WGAN aims to reduce mode collapse. *(source: medium)*

Training Challenges in GANs

Other Challenges

- **Non-convergence:** The model parameters may oscillate, become unstable, and never converge.

Training Challenges in GANs

Other Challenges

- **Non-convergence:** The model parameters may oscillate, become unstable, and never converge.
- **Vanishing Gradients:** If the discriminator gets too good too quickly, the generator's gradients can vanish, causing learning to stop.

Training Challenges in GANs

Other Challenges

- **Non-convergence:** The model parameters may oscillate, become unstable, and never converge.
- **Vanishing Gradients:** If the discriminator gets too good too quickly, the generator's gradients can vanish, causing learning to stop.
- **Hyperparameter Sensitivity:** GANs are very sensitive to the choice of hyperparameters, requiring significant fine-tuning.

Plausible solutions

Researchers have proposed many techniques to stabilize training and avoid mode collapse.

Techniques to Improve Stability and Diversity

- **Wasserstein GAN (WGAN)**: It measures the **Wasserstein distance** between the real and generated data distributions. By minimizing the distance between the entire distributions, the generator is strongly incentivized to produce a wide variety of outputs to match the diversity of the real data, thus significantly reducing mode collapse.

Plausible solutions

Researchers have proposed many techniques to stabilize training and avoid mode collapse.

Techniques to Improve Stability and Diversity

- **Wasserstein GAN (WGAN)**: It measures the **Wasserstein distance** between the real and generated data distributions. By minimizing the distance between the entire distributions, the generator is strongly incentivized to produce a wide variety of outputs to match the diversity of the real data, thus significantly reducing mode collapse.
- **Minibatch Discrimination:** Allows the discriminator to look at an entire batch of samples at once and measures their similarity. If a batch of fakes lacks diversity, it can be easily rejected, encouraging the generator to produce more varied outputs.

Plausible solutions

Researchers have proposed many techniques to stabilize training and avoid mode collapse.

Techniques to Improve Stability and Diversity

- **Wasserstein GAN (WGAN)**: It measures the **Wasserstein distance** between the real and generated data distributions. By minimizing the distance between the entire distributions, the generator is strongly incentivized to produce a wide variety of outputs to match the diversity of the real data, thus significantly reducing mode collapse.
- **Minibatch Discrimination:** Allows the discriminator to look at an entire batch of samples at once and measures their similarity. If a batch of fakes lacks diversity, it can be easily rejected, encouraging the generator to produce more varied outputs.
- **Progressive Growing of GANs:** Start by generating very small images (e.g., 4×4) and progressively add layers to both networks to generate larger images ($8 \times 8, 16 \times 16$, etc.). This simplifies the learning task at each stage.

Key Takeaways

- **Adversarial Training:** GANs consist of a Generator and a Discriminator locked in a competitive, zero-sum game, which drives the generator to produce increasingly realistic data.
- **Powerful Generative Models:** They are capable of generating very crisp and realistic images compared to other generative models.
- **Architecture Evolution:** The field has progressed rapidly from the original GAN to architectures like DCGANs, which established stable convolutional designs, and StyleGANs, which provide fine-grained control over the synthesis process.
- **Training is Challenging:** Issues like **mode collapse** and **training instability** are significant hurdles, though many techniques exist to mitigate them.
- **Meaningful Latent Space:** GANs can learn rich, meaningful latent representations that allow for semantic operations, such as vector arithmetic on faces.